ByteMart Digital

THE COMPLETE GUIDE

Spring Security

Authentication & Authorization with JWT and OAuth2

A complete, example-driven guide to securing Java applications with Spring Security — from the filter chain and authentication to JWT, OAuth2, OpenID Connect, and production hardening.

24	110+	11	6
Chapters	Code Examples	Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Spring Security: JWT & OAuth2

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Spring, Spring Security, OAuth, OpenID Connect, and other product names referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. Why Application Security 4
2. Spring Security Architecture 6
3. Configuring Spring Security 8
4. The Authentication Flow 11
5. UserDetails & UserDetailsService 13
6. Password Encoding 15
7. Form Login & HTTP Basic 17
8. The SecurityContext & Principal 19
9. Authorizing HTTP Requests 22
10. Method Security 24
11. Roles, Authorities & Expressions 27
12. Custom Filters & Entry Points 29
13. Stateless Auth & Why JWT 31
14. Anatomy of a JWT 33
15. Issuing JWTs 36
16. Validating JWTs 39
17. Refresh Tokens & Lifecycle 42
18. OAuth2 Concepts & Roles 45
19. The Authorization Code Flow 48
20. OAuth2 Login & OpenID Connect 51
21. The Resource Server 53
22. The OAuth2 Client 56
23. CORS, CSRF & Security Headers 58
24. Testing & Production Hardening 60

MODULE 1 · SPRING SECURITY FUNDAMENTALS

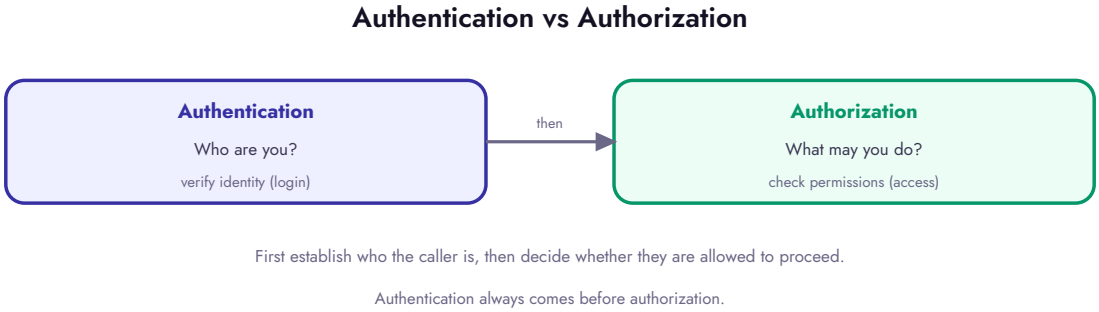
CHAPTER 01

Why Application Security

Authentication versus authorization, and the threats Spring Security defends against.

Almost every real application must answer two questions about each request: **who is calling**, and **are they allowed**. Spring Security is the framework that answers both, consistently and safely.

Authentication vs authorization



Authentication verifies identity; authorization decides what that identity may do.

Authentication establishes identity — verifying you are who you claim to be (a login). **Authorization** then decides what that identity is permitted to do. Authentication always precedes authorization.

> **Spring Security**
> A powerful, highly customisable framework for authentication and authorization in Java applications, built as a chain of servlet filters that secure every request into your Spring app.

What it protects against

- **Broken authentication** — weak logins, guessable credentials.
- **Broken access control** — users reaching data they should not.
- **Session and token attacks** — fixation, hijacking, replay.
- **Common web attacks** — CSRF, clickjacking, and more, handled by defaults.

Why not roll your own

> **WATCH OUT**
> Security is unforgiving — a subtle mistake becomes a breach. Spring Security encodes years of hard-won defaults and defends against attacks you may not even know exist. Use it rather than reinventing authentication.

> **Who, then what**
> Application security answers who is calling (authentication) and what they may do (authorization). Spring Security provides both, plus defences against common attacks — lean on it, do not rebuild it.

© 2026 ByteMart Digital

MODULE 1 · SPRING SECURITY FUNDAMENTALS

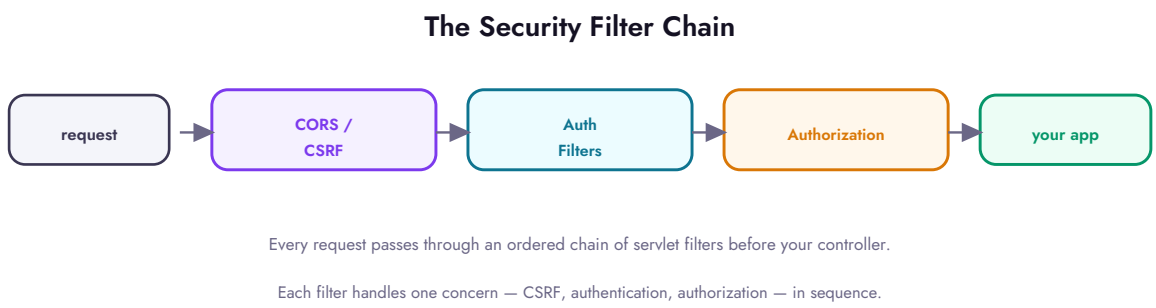
CHAPTER 02

Spring Security Architecture

The servlet filter chain that intercepts every request before it reaches your code.

Spring Security works by inserting a chain of **filters** in front of your application. Every request runs the gauntlet of these filters before a controller ever sees it — understanding this is the key to everything else.

The filter chain



Each request passes through an ordered chain of security filters before reaching your controller.

Spring Security registers a single `DelegatingFilterProxy` that delegates to a chain of security filters, each with one job — handling CSRF, extracting credentials, authenticating, and finally authorizing. Order matters, and Spring manages it for you.

Key filters

Filter	Responsibility
CsrfFilter	CSRF protection
Authentication filters	extract & verify credentials
BearerTokenAuthenticationFilter	read JWT bearer tokens
AuthorizationFilter	enforce access rules

How authentication and authorization split

Early filters **authenticate** — they establish the caller's identity and store it. A later filter **authorizes** — it checks that identity against the rules for the requested resource. If either fails, the request is rejected before your code runs.

> **Filters guard the door**
> Spring Security is a chain of ordered servlet filters in front of your app. Early filters authenticate the caller; a later filter authorizes the request. Everything you configure customises this chain.

MODULE 1 · SPRING SECURITY FUNDAMENTALS

CHAPTER 03

Configuring Spring Security

The modern, component-based SecurityFilterChain bean and the lambda DSL.

Modern Spring Security (6.x, on Spring Boot 3) is configured with a `SecurityFilterChain` bean and a fluent lambda DSL. The old `WebSecurityConfigurerAdapter` is gone — configuration is now component-based.

A SecurityFilterChain bean

java

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

Reading the DSL

- `authorizeHttpRequests` — declare access rules for URL patterns.
- `requestMatchers` — match paths (replaces the old antMatchers).
- `permitAll` / `authenticated` / `hasRole` — the access decision.
- `anyRequest` — the catch-all, placed last.

WATCH OUT

Order matters in `authorizeHttpRequests`: rules are evaluated top to bottom, and the first match wins. Put specific matchers before `anyRequest()`, which must come last.

Multiple chains

You can define several `SecurityFilterChain` beans, each with a `securityMatcher`, to apply different rules to different parts of your app — for example a stateless JWT chain for `/api` and a form-login chain for the web UI.

© 2026 ByteMart Digital

Configure with a bean and DSL

Define a SecurityFilterChain bean and use the lambda DSL to declare access rules. Rules match top-to-bottom with anyRequest last, and multiple chains can secure different parts of the app differently.

© 2026 ByteMart Digital

MODULE 1 · SPRING SECURITY FUNDAMENTALS

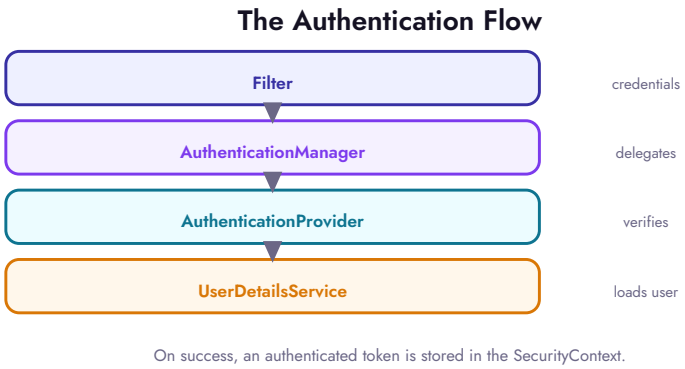
CHAPTER 04

The Authentication Flow

How credentials travel from a filter through the manager, provider, and back as an authenticated token.

When someone logs in, their credentials pass through a well-defined chain of collaborators. Knowing this flow lets you plug in custom logic at exactly the right point.

The collaborators



A filter hands credentials to the AuthenticationManager, which delegates to a provider that loads and verifies the user.

Component	Role
Authentication	holds credentials, then the result
AuthenticationManager	entry point; delegates
AuthenticationProvider	performs the actual check
UserDetailsService	loads the user by username

The sequence

A filter builds an unauthenticated `Authentication` from the credentials and passes it to the `AuthenticationManager`. The manager delegates to an `AuthenticationProvider` — commonly `DaoAuthenticationProvider` — which loads the user via `UserDetailsService` and checks the password with the `PasswordEncoder`.

The result

On success, an **authenticated** `Authentication` (with the principal and authorities) is returned and stored in the `SecurityContext`. On failure, an `AuthenticationException` is thrown and the request is rejected.

> **Credentials in, token out**
> A filter passes credentials to the AuthenticationManager, which delegates to a provider that loads the user and verifies the password. Success yields an authenticated token stored in the SecurityContext.

© 2026 ByteMart Digital

MODULE 2 · CORE AUTHENTICATION

CHAPTER 05

UserDetails & UserDetailsService

Telling Spring Security who your users are and how to load them.

Spring Security does not know about your user table — you tell it. Two abstractions bridge the gap: `UserDetails` models a user, and `UserDetailsService` loads one by username.

UserDetails

`UserDetails` is Spring Security's view of a user: username, (encoded) password, authorities, and account status flags. You can implement it on your own entity or adapt to it.

UserDetailsService

Implement `UserDetailsService` to load a user by username from your database. The authentication provider calls it during login.

java

```
@Service
public class DbUserDetailsService implements UserDetailsService {
    private final UserRepository repo;

    @Override
    public UserDetails loadUserByUsername(String username) {
        User u = repo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));
        return org.springframework.security.core.userdetails.User
            .withUsername(u.getUsername())
            .password(u.getPasswordHash())
            .authorities(u.getRoles().toArray(new String[0]))
            .build();
    }
}
```

WATCH OUT

Never reveal whether it was the username or the password that was wrong. Throw the same generic authentication failure for both, so attackers cannot enumerate valid usernames.

Wiring it up

With a `UserDetailsService` bean and a `PasswordEncoder` bean present, Spring Boot auto-configures a `DaoAuthenticationProvider` — no further wiring needed for standard database login.

Model and load your users

UserDetails describes a user to Spring Security; UserDetailsService loads one by username from your store. Provide both (plus a PasswordEncoder) and database login works automatically.

MODULE 2 · CORE AUTHENTICATION

CHAPTER 06

Password Encoding

Storing passwords safely with adaptive hashing, never in plain text.

Passwords must never be stored in plain text — or even with a fast hash. Spring Security uses adaptive, salted hashing through the `PasswordEncoder` abstraction, with BCrypt as the default.

The PasswordEncoder

```java
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

// hashing on registration
String hash = passwordEncoder.encode(rawPassword);
// checking on login (Spring does this for you)
boolean ok = passwordEncoder.matches(rawPassword, storedHash);
```

Why BCrypt (or Argon2)

BCrypt is **adaptive** — deliberately slow and tunable, so brute-forcing hashes stays expensive as hardware improves. It salts each password automatically, defeating rainbow tables. Argon2 and SCrypt are strong modern alternatives.

WATCH OUT

Never use plain MD5 or SHA-256 for passwords — they are fast, which is exactly wrong for password hashing. Fast hashes let attackers try billions of guesses per second.

DelegatingPasswordEncoder

The default Spring Boot encoder is a `DelegatingPasswordEncoder`: it prefixes each hash with its algorithm (like `{bcrypt}`), so you can upgrade algorithms over time and verify old hashes while writing new ones with a stronger scheme.

```java
PasswordEncoder encoder =
    PasswordEncoderFactories.createDelegatingPasswordEncoder();
```

Hash slow, salt always

Store passwords with an adaptive, salted encoder like BCrypt or Argon2 — never plain text or fast hashes. The DelegatingPasswordEncoder lets you evolve algorithms without breaking existing users.

© 2026 ByteMart Digital

MODULE 2 · CORE AUTHENTICATION

CHAPTER 07

Form Login & HTTP Basic

The two built-in authentication mechanisms, and when each fits.

Before tokens, it helps to know the two authentication styles Spring Security supports out of the box: form login for browsers, and HTTP Basic for simple API access.

Form login

`formLogin` presents a login page, accepts a username and password, and establishes a **session**. It suits traditional server-rendered web applications where the browser holds a session cookie.

```java
http.formLogin(form -> form
    .loginPage("/login").permitAll());
```

HTTP Basic

`httpBasic` reads credentials from the `Authorization` header on every request. It is simple and stateless-ish, useful for machine-to-machine calls, but it sends credentials on each request and needs HTTPS.

```java
http.httpBasic(Customizer.withDefaults());
```

Why neither fits a modern API

- Form login relies on server-side **sessions**, which do not scale statelessly.
- HTTP Basic resends credentials constantly and cannot easily carry roles or expiry.
- Neither suits SPAs and mobile clients calling a REST API across services.

NOTE

These mechanisms are perfect for their niches, but modern REST APIs, SPAs, and microservices favour stateless tokens — which is where JWT comes in, later in this guide.

Sessions for browsers, tokens for APIs

Form login (session-based) fits server-rendered web apps; HTTP Basic fits simple machine calls over HTTPS. Neither scales well for stateless REST APIs — the motivation for JWT.

© 2026 ByteMart Digital

MODULE 2 · CORE AUTHENTICATION

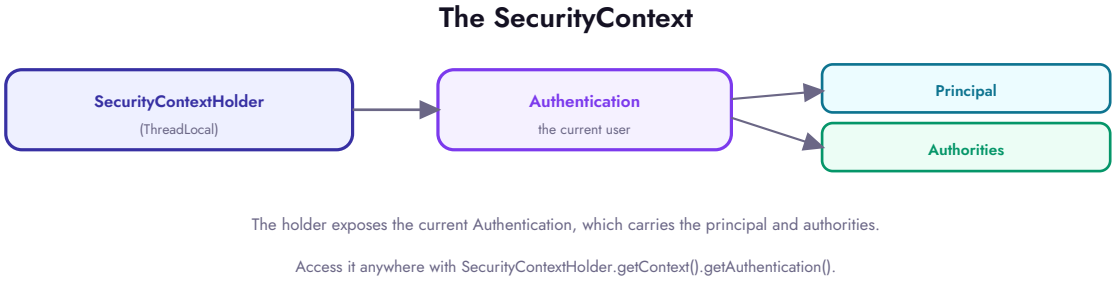
CHAPTER 08

The SecurityContext & Principal

Where the authenticated user lives, and how to access it in your code.

Once a request is authenticated, the resulting identity must be reachable throughout the request. That is the job of the `SecurityContext` — the home of the current user.

The structure



The holder exposes the Authentication, which carries the principal and the granted authorities.

`SecurityContextHolder` holds the `SecurityContext`, which holds the `Authentication`. The `Authentication` carries the **principal** (the user) and the **granted authorities** (roles and permissions).

Accessing the current user

```java
Authentication auth =
    SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
Collection<? extends GrantedAuthority> roles = auth.getAuthorities();
```

More conveniently, inject it into controllers with `@AuthenticationPrincipal` or a `Principal` parameter.

```java
@GetMapping("/me")
String me(@AuthenticationPrincipal UserDetails user) {
    return user.getUsername();
}
```

Thread-bound by default

The context is stored in a `ThreadLocal`, so it is available anywhere on the request thread but does not automatically cross to new threads. (Reactive apps use the Reactor Context instead — see ByteMart's WebFlux guide.)

The current user lives here

SecurityContextHolder exposes the Authentication, carrying the principal and authorities. Access it directly or via @AuthenticationPrincipal — it is thread-bound on the request thread.

© 2026 ByteMart Digital

MODULE 3 · AUTHORIZATION

CHAPTER 09

Authorizing HTTP Requests

Declaring which users can reach which URLs with request matchers.

09

The most common authorization is URL-based: some paths are public, some need a login, some need a specific role. You declare this with `authorizeHttpRequests` and request matchers.

Matching and deciding

java

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/", "/public/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
    .requestMatchers("/api/orders/**").hasRole("CUSTOMER")
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated());
```

The access decisions

Method	Grants access to
permitAll()	everyone
authenticated()	any logged-in user
hasRole("X")	users with role X
hasAuthority("p")	users with authority p
denyAll()	no one

Matching by method and path

Matchers can restrict by HTTP method and path pattern, so you can allow anyone to `GET` products but require a role to `POST` one. Combine matchers to express precise rules.

WATCH OUT

Rules are evaluated in order, first match wins, and `anyRequest()` must be last. A misplaced broad matcher can accidentally expose protected paths — order carefully.

URL rules, top to bottom

authorizeHttpRequests maps request matchers to access decisions (permitAll, authenticated, hasRole). Match by method and path, order specific-to-general, and end with anyRequest().

© 2026 ByteMart Digital

MODULE 3 · AUTHORIZATION

CHAPTER 10

Method Security

Securing service methods with annotations for fine-grained, expression-based rules.

URL rules protect endpoints, but sometimes you want authorization closer to the logic — on service methods. **Method security** uses annotations with rich expressions to do exactly that.

Enabling it

java

```
@Configuration
@EnableMethodSecurity   // turns on @PreAuthorize etc.
public class MethodSecurityConfig {}
```

@PreAuthorize and @PostAuthorize

`@PreAuthorize` checks a condition **before** the method runs; `@PostAuthorize` checks **after**, and can inspect the return value. Both use the Spring Security expression language.

java

```
@PreAuthorize("hasRole('ADMIN')")
void deleteUser(Long id) { ... }

@PreAuthorize("#userId == authentication.name or hasRole('ADMIN')")
User getProfile(String userId) { ... }

@PostAuthorize("returnObject.owner == authentication.name")
Document load(Long id) { ... }
```

Expression power

- Reference method arguments with `#argName`.
- Reference the current user via `authentication` and `principal`.
- Combine checks with `and`, `or`, `not`.
- Inspect the result with `returnObject` in `@PostAuthorize`.

TIP

Method security expresses **ownership** rules cleanly — 'you may edit only your own resource' — which URL rules alone cannot capture.

Authorize at the method

@EnableMethodSecurity plus @PreAuthorize/@PostAuthorize secures service methods with expressions referencing arguments, the current user, and even the return value — ideal for ownership and fine-grained rules.

© 2026 ByteMart Digital

MODULE 3 · AUTHORIZATION

CHAPTER 11

Roles, Authorities & Expressions

The model behind access decisions, and the subtle role-versus-authority distinction.

Every authorization decision comes down to the authorities the current user holds. Understanding **roles** versus **authorities** — and the `ROLE_` convention — prevents a classic source of bugs.

Authorities and roles

A **granted authority** is any permission string a user holds. A **role** is simply an authority with the `ROLE_` prefix by convention. `hasRole("ADMIN")` checks for the authority `ROLE_ADMIN`.

Check	Looks for authority
hasRole("ADMIN")	ROLE_ADMIN
hasAuthority("ROLE_ADMIN")	ROLE_ADMIN
hasAuthority("orders:read")	orders:read

WATCH OUT

The single most common Spring Security mistake: calling `hasRole("ROLE_ADMIN")`. `hasRole` adds the `ROLE_` prefix itself, so this looks for `ROLE_ROLE_ADMIN` and never matches.

Roles vs fine-grained permissions

Roles (`ADMIN`, `CUSTOMER`) are coarse and simple. Fine-grained authorities (`orders:read`, `orders:write`) scale better as permissions multiply. Many systems assign permissions to roles and grant users roles — the best of both.

Expressions

The same expression language drives URL rules and method security: `hasRole`, `hasAuthority`, `hasAnyRole`, `authentication`, `principal`, and boolean combinations — a consistent model everywhere.

Roles are prefixed authorities

A role is an authority with the ROLE_ prefix; hasRole adds it automatically (so never write ROLE_ yourself in hasRole). Use roles for coarse access and fine-grained authorities as permissions grow.

MODULE 3 · AUTHORIZATION

CHAPTER 12

Custom Filters & Entry Points

Extending the chain with your own filter, and controlling failure responses.

Sometimes the built-in filters are not enough — you need custom authentication (like reading a JWT). Spring Security lets you insert your own filter and control what happens when authentication or authorization fails.

A custom filter

Extend `OncePerRequestFilter` to run your logic once per request, then register it at the right position in the chain. This is exactly how JWT authentication is added, as you will see next.

java

```
public class ApiKeyFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String key = req.getHeader("X-API-Key");
        // validate, build Authentication, set on the context...
        chain.doFilter(req, res);
    }
}
```

java

```
http.addFilterBefore(new ApiKeyFilter(),
    UsernamePasswordAuthenticationFilter.class);
```

Entry points and handlers

Component	Handles
AuthenticationEntryPoint	unauthenticated (401) responses
AccessDeniedHandler	authenticated-but-forbidden (403)

For a REST API you typically configure these to return clean JSON with `401` or `403` status codes, rather than redirecting to a login page.

Extend the chain, shape failures

Add custom logic with a OncePerRequestFilter placed at the right position, and customise the AuthenticationEntryPoint (401) and AccessDeniedHandler (403) to return API-friendly responses.

© 2026 ByteMart Digital

MODULE 4 · JWT AUTHENTICATION

CHAPTER 13

Stateless Auth & Why JWT

The problem with server-side sessions at scale, and how tokens solve it.

Session-based login stores state on the server — which becomes a burden when you run many instances. **JWT** (JSON Web Token) makes authentication **stateless** by putting the identity into a signed token the client carries.

Sessions vs tokens



Sessions keep state on the server; JWTs carry signed state the server only needs to verify.

With sessions, the server stores each logged-in user and the client holds a session id. With JWT, the server issues a signed token containing the identity; on each request the client sends it, and the server just verifies the signature — no lookup, no stored state.

Why stateless wins for APIs

- **Horizontal scaling** — any instance can validate any request; no shared session store.
- **Cross-service** — the same token works across microservices.
- **Client-agnostic** — SPAs and mobile apps carry the token naturally.

The trade-off

WATCH OUT

Statelessness has a cost: a JWT cannot be easily revoked before it expires, because the server keeps no record of it. Keep access tokens short-lived and design for this — covered in the refresh-token chapter.

Carry identity in a signed token

JWT makes authentication stateless: the client presents a signed token and the server verifies it without storing sessions. This scales across instances and services — at the cost of harder revocation.

MODULE 4 · JWT AUTHENTICATION

CHAPTER 14

Anatomy of a JWT

The three parts of a token, what belongs in the claims, and what signing actually guarantees.

A JWT is three Base64url-encoded parts joined by dots: a header, a payload of claims, and a signature. Understanding each — especially what signing does and does not do — is essential to using them safely.

The three parts



A JWT is header.payload.signature — the first two are encoded, the third proves integrity.

Part	Contains
Header	algorithm and token type
Payload	claims: subject, expiry, roles
Signature	proof of integrity and origin

Standard claims

The payload carries **claims** — statements about the user. Registered claims include `sub` (subject/user), `iss` (issuer), `exp` (expiry), and `iat` (issued-at). You add custom claims like roles.

Encoded, not encrypted

WATCH OUT

The header and payload are only Base64url-**encoded**, not encrypted — anyone with the token can read the claims. Never put secrets or sensitive data in a JWT payload.

What the signature guarantees

The signature is computed over the header and payload using a secret (HMAC) or private key (RSA/EC). It proves the token was issued by you and has not been altered — if a single byte of the claims changes, the signature no longer verifies.

© 2026 ByteMart Digital

Read the claims, trust the signature

A JWT is header.payload.signature. The payload is readable (never store secrets in it); the signature is what proves the token is authentic and untampered. Always set an expiry claim.

© 2026 ByteMart Digital

MODULE 4 · JWT AUTHENTICATION

CHAPTER 15

Issuing JWTs

Building a login endpoint that authenticates the user and signs a token.

To issue tokens you need a login endpoint: authenticate the credentials, then sign a JWT with the user's identity and roles. Spring Security's `JwtEncoder` makes the signing idiomatic.

Authenticate, then sign

Verify the credentials through the `AuthenticationManager`, then build claims and encode a signed token.

java

```java
@PostMapping("/login")
public TokenResponse login(@RequestBody LoginRequest req) {
    Authentication auth = authManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            req.username(), req.password()));

    Instant now = Instant.now();
    JwtClaimsSet claims = JwtClaimsSet.builder()
        .issuer("bytemart")
        .issuedAt(now)
        .expiresAt(now.plus(15, ChronoUnit.MINUTES))
        .subject(auth.getName())
        .claim("roles", authoritiesToStrings(auth))
        .build();

    String token = jwtEncoder.encode(
        JwtEncoderParameters.from(claims)).getTokenValue();
    return new TokenResponse(token);
}
```

Signing keys: HMAC vs RSA

Approach	Key	When
HMAC (HS256)	shared secret	one service issues & verifies
RSA (RS256)	private / public	others verify with the public key

Use RSA when separate resource servers must verify tokens — they need only the public key, never the signing secret. Configure a `JwtEncoder` (Nimbus) bean with your key.

WATCH OUT

Keep the signing key secret and strong. Anyone with an HMAC secret — or an RSA private key — can forge valid tokens for any user.

Verify credentials, then sign claims

A login endpoint authenticates via the AuthenticationManager, builds claims (subject, expiry, roles), and signs a JWT with a JwtEncoder. Use RSA when other services must verify with a public key.

MODULE 4 · JWT AUTHENTICATION

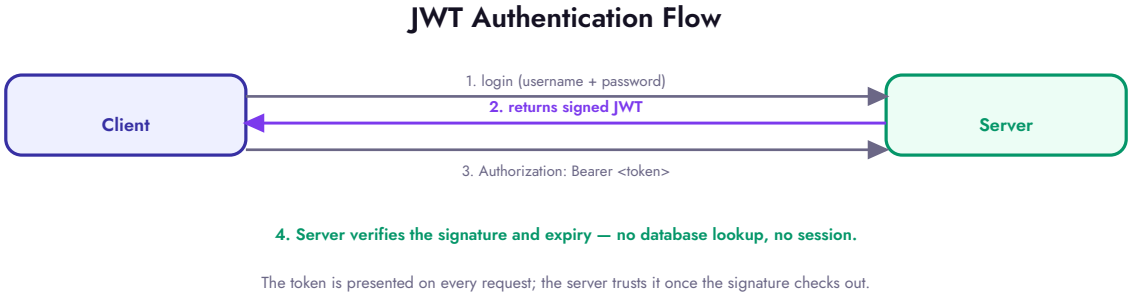
CHAPTER 16

Validating JWTs

Turning your API into a resource server that authenticates every request from its token.

Issuing tokens is half the story; the other half is validating them on every request. The cleanest way in Spring Security is to configure your API as an OAuth2 **resource server** with JWT support.

The request flow



Client logs in, receives a JWT, and sends it as a Bearer token; the server verifies signature and expiry.

Resource server configuration

Enable JWT resource-server support and provide a `JwtDecoder`. Spring Security's `BearerTokenAuthenticationFilter` then extracts, validates, and authenticates every request automatically.

```java
http
    .sessionManagement(s -> s
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/login").permitAll()
        .anyRequest().authenticated())
    .oauth2ResourceServer(oauth2 -> oauth2
        .jwt(Customizer.withDefaults()));
```

The decoder

The `JwtDecoder` checks the signature and standard claims (like expiry). For HMAC you build it from the secret; for RSA from the public key or a JWK Set URL. Validation happens on the server with no database call.

java

```
@Bean
JwtDecoder jwtDecoder() {
    return NimbusJwtDecoder.withPublicKey(rsaPublicKey).build();
}
```

Mapping claims to authorities

By default Spring maps `scope/scp` claims to authorities. For a custom `roles` claim, provide a `JwtAuthenticationConverter` so your roles drive `hasRole` checks.

Let the resource server validate

Configure oauth2ResourceServer().jwt() with a JwtDecoder and a stateless session policy. Spring Security then validates the Bearer token's signature and expiry on every request — no sessions, no lookups.

MODULE 4 · JWT AUTHENTICATION

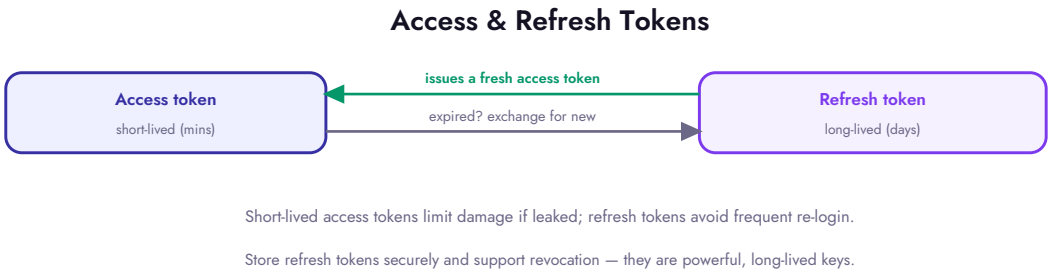
CHAPTER 17

Refresh Tokens & Lifecycle

Balancing security and convenience with short access tokens and longer refresh tokens.

A JWT cannot be revoked before it expires, so you want access tokens to be short-lived. But forcing a re-login every few minutes is terrible UX. **Refresh tokens** resolve this tension.

Two tokens, two lifetimes



A short-lived access token is exchanged — via a long-lived refresh token — for a fresh one when it expires.

Token	Lifetime	Used for
Access	minutes	calling APIs
Refresh	days/weeks	getting new access tokens

The flow

On login the client receives both tokens. It uses the access token until it expires, then presents the refresh token to a dedicated endpoint to obtain a new access token — without re-entering credentials.

Making refresh tokens revocable

Because refresh tokens are long-lived and powerful, treat them differently from access tokens: store them server-side (so they can be revoked), rotate them on each use, and bind them to the user and device.

- **Store & track** refresh tokens so you can revoke on logout or compromise.
- **Rotate** — issue a new refresh token each time, invalidating the old.
- **Detect reuse** — a replayed old refresh token signals theft; revoke the family.

WATCH OUT

A leaked long-lived refresh token is a serious breach. Store them securely (HttpOnly cookie or secure storage), rotate them, and always support revocation.

© 2026 ByteMart Digital

Short access, revocable refresh

Keep access tokens short-lived and non-revocable, and use longer refresh tokens — stored, rotated, and revocable — to mint new ones. This balances stateless scale with security and good UX.

© 2026 ByteMart Digital

MODULE 5 · OAUTH2 & OPENID CONNECT

CHAPTER 18

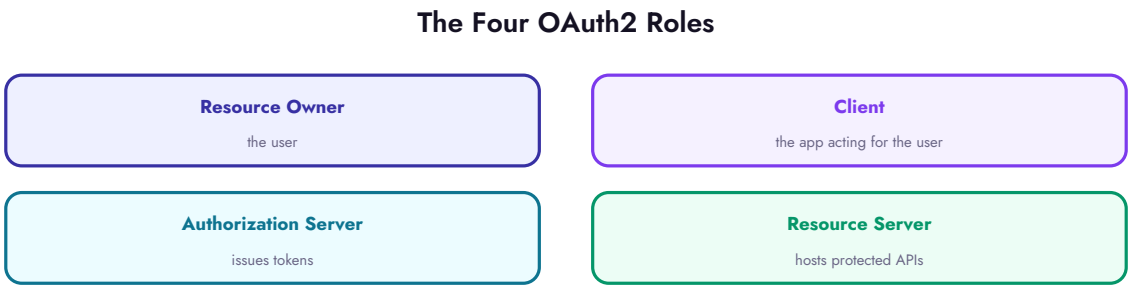
OAuth2 Concepts & Roles

The delegated-authorization model and its four roles, and how it differs from plain JWT auth.

OAuth2 is not a login mechanism — it is a framework for **delegated authorization**: letting an application access resources on a user's behalf without handling their password. Its four roles make the model precise.

> **OAuth2**
> An authorization framework that lets a third-party application obtain limited access to a resource on behalf of a user, via tokens issued by an authorization server — without the app ever seeing the user's credentials.

The four roles



The client gets a token from the authorization server to access the resource server on behalf of the resource owner — without ever seeing their password.

The client gets a token from the authorization server to reach the resource server for the user.

Role	Who / what
Resource Owner	the user who owns the data
Client	the app acting on their behalf
Authorization Server	authenticates and issues tokens
Resource Server	hosts the protected APIs

Access tokens and scopes

The authorization server issues an **access token** — often a JWT — that the client presents to the resource server. **Scopes** limit what the token can do (`read`, `write`), so access is least-privilege.

OAuth2 vs plain JWT auth

In the earlier chapters your app both issued and validated its own JWTs. OAuth2 introduces a dedicated **authorization server** (yours or a provider like Google) that issues tokens for potentially many clients and resource servers — a cleaner separation at scale.

© 2026 ByteMart Digital

Delegated access, four roles

OAuth2 lets a client access resources for a user via tokens from an authorization server, without touching the password. The resource owner, client, authorization server, and resource server are its four roles.

MODULE 5 · OAUTH2 & OPENID CONNECT

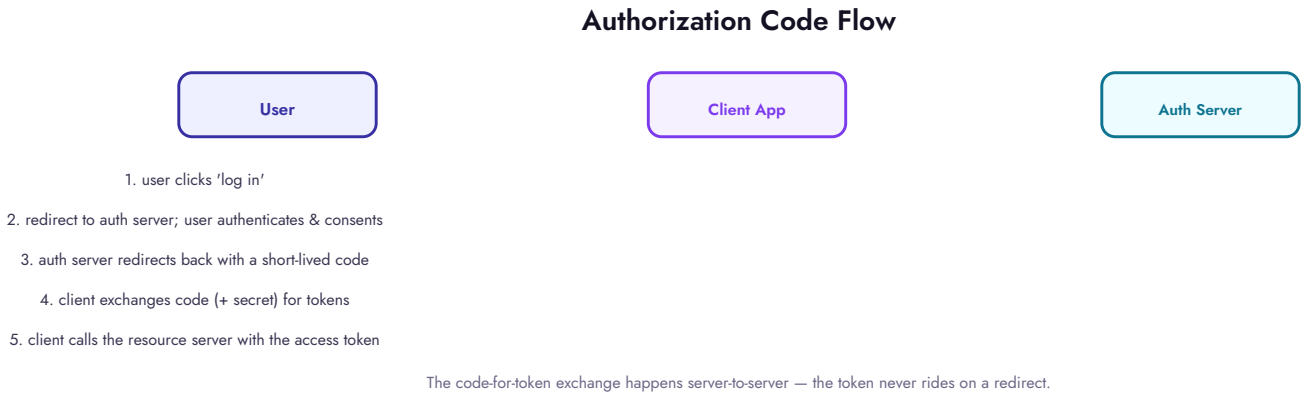
CHAPTER 19

The Authorization Code Flow

The standard, secure grant for user-facing applications — step by step.

OAuth2 defines several **grant types**. For applications acting on behalf of a user, the **Authorization Code** flow is the standard and most secure choice — and the one you will use most.

The flow, step by step



The user authenticates at the auth server; the client swaps a short-lived code for tokens server-to-server.

The key insight: the client never sees the user's password, and the access token is obtained through a back-channel exchange rather than being exposed in a browser redirect. A short-lived **code** rides the redirect; the client swaps it (with its secret) for tokens server-to-server.

Why the code step exists

Sending the token directly on the redirect would expose it in browser history and logs. The one-time code, exchanged securely by the backend, keeps the actual token off the front channel entirely.

PKCE for public clients

Single-page and mobile apps cannot keep a client secret. **PKCE** (Proof Key for Code Exchange) adds a one-time cryptographic challenge to the flow so the code cannot be intercepted and reused — now recommended for all clients.

Other grants, briefly

- **Client Credentials** — machine-to-machine, no user involved.
- **Authorization Code + PKCE** — user-facing apps (the default).
- The legacy **Password** and **Implicit** grants are discouraged.

© 2026 ByteMart Digital

Code first, token by back-channel

The Authorization Code flow authenticates the user at the auth server, returns a short-lived code on the redirect, and exchanges it for tokens server-to-server — with PKCE protecting public clients.

MODULE 5 · OAUTH2 & OPENID CONNECT

CHAPTER 20

OAuth2 Login & OpenID Connect

Adding social and enterprise login with a few lines using OIDC.

'Log in with Google' is OAuth2 plus **OpenID Connect**. OIDC is an identity layer on top of OAuth2 that adds authentication — and Spring Security supports it with remarkably little code.

OIDC in one sentence

OAuth2 handles authorization (access to resources); **OIDC** adds an `ID token` that proves who the user is. Together they let a provider authenticate your users for you.

Enabling OAuth2 login

java

```
http
    .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
    .oauth2Login(Customizer.withDefaults());
```

Configuring a provider

Register the client with its provider details in properties. Spring Boot auto-configures the redirect flow; the user clicks login, authenticates at the provider, and returns authenticated.

properties

```
spring.security.oauth2.client.registration.google.client-id=${GOOGLE_ID}
spring.security.oauth2.client.registration.google.client-secret=${GOOGLE_SECRET}
spring.security.oauth2.client.registration.google.scope=openid,profile,email
```

Accessing the user

After login, inject the authenticated principal as an `OidcUser` to read profile claims (name, email) provided by the identity provider.

java

```
@GetMapping("/me")
String me(@AuthenticationPrincipal OidcUser user) {
    return user.getEmail();
}
```

Delegate login with OIDC

OpenID Connect adds authentication to OAuth2 via an ID token. Enable oauth2Login and configure a provider, and Spring Security handles the whole social/enterprise login flow for you.

© 2026 ByteMart Digital

MODULE 5 · OAUTH2 & OPENID CONNECT

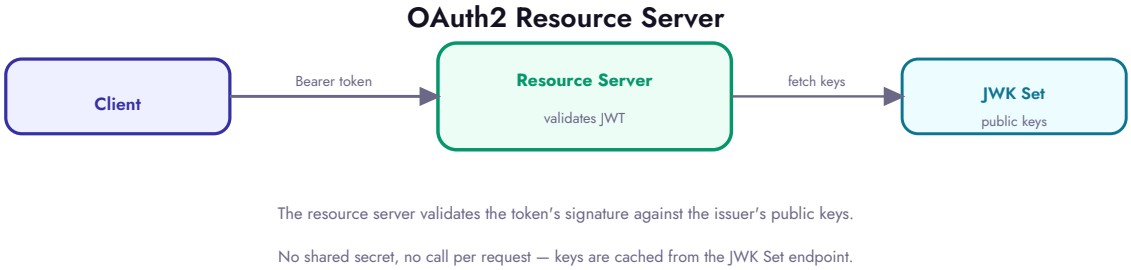
CHAPTER 21

The Resource Server

Protecting an API with tokens issued by an external authorization server.

When tokens come from a separate authorization server — your own or a provider like Keycloak, Auth0, or Okta — your API becomes a pure **resource server** that only validates them.

Validating with the issuer



The resource server validates the token's signature against the issuer's public keys.

No shared secret, no call per request — keys are cached from the JWK Set endpoint.

The resource server validates the token's signature against the issuer's published public keys.

You point the resource server at the authorization server's issuer URI. Spring Security discovers its **JWK Set** (public keys), caches them, and validates each token's signature and claims — no shared secret, no per-request call to the auth server.

```properties
spring.security.oauth2.resourceserver.jwt.issuer-uri=\
  https://auth.bytemart.com/realms/bytemart
```

```java
http.oauth2ResourceServer(oauth2 ->
    oauth2.jwt(Customizer.withDefaults()));
```

Scopes and authorities

Tokens from an authorization server carry **scopes**. Spring maps them to authorities like `SCOPE_read`, which you enforce with `hasAuthority("SCOPE_read")` on endpoints or methods.

```java
.requestMatchers(HttpMethod.POST, "/api/**")
    .hasAuthority("SCOPE_write")
```

Why this scales

Many resource servers can validate tokens from one authorization server using only its public keys. Add a new service and it trusts existing tokens instantly — the model that powers microservice security.

Validate against the issuer

A resource server points at the authorization server's issuer URI, fetches its public keys, and validates JWTs locally — enforcing scopes as authorities. One auth server can secure many services.

© 2026 ByteMart Digital

MODULE 5 · OAUTH2 & OPENID CONNECT

CHAPTER 22

The OAuth2 Client

Calling protected downstream APIs on behalf of the user or the application.

The flip side of a resource server is a **client** — your app calling another protected API. Spring Security manages obtaining, storing, and attaching the right token for you.

Two client scenarios

- **On behalf of a user** — use the user's token (from OAuth2 login) to call a downstream API.
- **On behalf of the app** — use the **client credentials** grant for machine-to-machine calls with no user.

Attaching tokens automatically

Configure a `WebClient` (or `RestClient`) with Spring Security's OAuth2 support, and it obtains and attaches the access token to outbound requests — refreshing it as needed.

java

```
String data = webClient.get()
    .uri("https://api.partner.com/data")
    .attributes(clientRegistrationId("partner"))
    .retrieve()
    .bodyToMono(String.class)
    .block();
```

Client credentials for services

For backend-to-backend calls with no user, the client-credentials grant obtains a token using the app's own identity. Spring Security's authorized-client manager handles acquiring and caching it.

TIP

Let Spring manage token acquisition and refresh through the authorized-client abstraction — manually handling tokens on outbound calls is error-prone and easy to get wrong.

Let Spring carry the token

As an OAuth2 client, your app calls downstream APIs using the user's token or the client-credentials grant. Configure WebClient with OAuth2 support and Spring obtains, attaches, and refreshes tokens for you.

© 2026 ByteMart Digital

MODULE 6 · PRODUCTION

CHAPTER 23

CORS, CSRF & Security Headers

The browser-facing defences — and why stateless APIs treat CSRF differently.

Beyond authentication, Spring Security defends against a family of web attacks. Three areas deserve explicit attention: CORS, CSRF, and response headers — and their correct settings differ for stateless APIs.

CSRF

Cross-Site Request Forgery tricks a logged-in browser into making unwanted requests using its **cookies**. Spring Security enables CSRF protection by default for session-based apps.

WATCH OUT

For a **stateless JWT API**, tokens are sent in the `Authorization` header, not cookies, so CSRF does not apply and is commonly disabled. But if you store the token in a cookie, CSRF protection is back on the table — the decision depends on where the token lives.

java

```
// typical for a stateless bearer-token API
http.csrf(csrf -> csrf.disable());
```

CORS

Cross-Origin Resource Sharing controls which browser origins may call your API. A SPA on a different origin needs an explicit, **narrow** CORS policy — specific origins and methods, never a blanket wildcard on a credentialed API.

java

```
http.cors(Customizer.withDefaults());
// backed by a CorsConfigurationSource bean with allowed origins
```

Security headers

Spring Security sets protective response headers by default — disabling caching of secured pages, preventing clickjacking (`X-Frame-Options`), and enabling HSTS over HTTPS. Add a Content-Security-Policy for defence in depth.

Match defences to the client

Enable CSRF for cookie/session apps and disable it for header-based stateless APIs; configure a narrow CORS policy for cross-origin SPAs; and keep Spring's secure headers on, adding a CSP.

© 2026 ByteMart Digital

MODULE 6 · PRODUCTION

CHAPTER 24

Testing & Production Hardening

Testing secured code and a final checklist for shipping authentication safely.

Security code must be tested like any other — and shipped with discipline. This closing chapter covers testing with Spring Security's helpers and a practical hardening checklist.

Testing secured endpoints

The `spring-security-test` module lets you run requests as a given user or with a given token, so you can assert both allowed and forbidden access.

```java
@Test
@WithMockUser(roles = "ADMIN")
void adminCanDelete() throws Exception {
    mvc.perform(delete("/admin/users/1"))
       .andExpect(status().isOk());
}

@Test
void anonymousIsUnauthorized() throws Exception {
    mvc.perform(get("/api/orders"))
       .andExpect(status().isUnauthorized());
}
```

For resource-server tests, post-process the request with a mock JWT and its authorities to simulate a valid token.

Production checklist

- Serve everything over **HTTPS** — tokens and credentials must never cross plaintext.
- Keep access tokens **short-lived**; make refresh tokens revocable and rotated.
- Protect signing keys; rotate them, and prefer RSA when others verify.
- Hash passwords with BCrypt/Argon2; return generic login failures.
- Apply **least privilege** with scopes and fine-grained authorities.
- Set correct CORS, CSRF, and security headers for your client model.
- Log authentication events and rate-limit login to slow brute force.

WATCH OUT

The most damaging mistakes are boring ones: tokens over HTTP, long-lived non-revocable tokens, a leaked signing key, and overly broad access rules. Get these right before anything clever.

© 2026 ByteMart Digital

Test it, then harden it

Use spring-security-test to assert both allowed and denied access, and ship against a checklist: HTTPS everywhere, short revocable tokens, protected keys, strong password hashing, least privilege, and correct headers.

© 2026 ByteMart Digital